

Companion to Bird & de Moor, Algebra of Programming

$$\begin{array}{c} X \\ \hline \frac{1}{\exists} (\epsilon \backslash R) \end{array} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \begin{array}{c} \frac{1}{\exists}^\circ X \\ \hline \epsilon \backslash R \end{array} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \begin{array}{c} \epsilon \frac{1}{\exists}^\circ X \\ \hline R \end{array} \xleftrightarrow{\circ} \begin{array}{c} (\frac{1}{\exists} \exists)^\circ X \\ \hline R \end{array} \xleftrightarrow{i \cdot \neg E} \begin{array}{c} X \\ \hline R \end{array} \quad (7.1)$$

§7.1.3. $\frac{S}{\exists} (\epsilon \backslash R) = S^\circ \backslash R$

$\frac{S}{\exists}$ gathers the S -image of a point into one set and $\epsilon \backslash R$ asks that every member of that set be R -related to the target, so the set cancels and $S^\circ \backslash R$ asks it of the S -image directly.

$$\begin{array}{c} X \\ \hline \frac{S}{\exists} (\epsilon \backslash R) \end{array} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \begin{array}{c} \frac{S}{\exists}^\circ X \\ \hline \epsilon \backslash R \end{array} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \begin{array}{c} \epsilon \frac{S}{\exists}^\circ X \\ \hline R \end{array} \xleftrightarrow{\circ} \begin{array}{c} (\frac{S}{\exists} \exists)^\circ X \\ \hline R \end{array} \xleftrightarrow{\cdot \exists \neg \frac{S}{\exists}} \begin{array}{c} S^\circ X \\ \hline R \end{array} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \begin{array}{c} X \\ \hline S^\circ \backslash R \end{array} \quad (7.2)$$

§7.1.4. **union** $(\epsilon \backslash R) = \epsilon \backslash (\epsilon \backslash R)$

$$\begin{array}{c} X \\ \hline \text{union} (\epsilon \backslash R) \end{array} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \begin{array}{c} \text{union}^\circ X \\ \hline \epsilon \backslash R \end{array} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \begin{array}{c} \epsilon \text{union}^\circ X \\ \hline R \end{array} \xleftrightarrow{\circ} \begin{array}{c} (\text{union} \exists)^\circ X \\ \hline R \end{array} \xleftrightarrow{\cdot \exists \neg \frac{S}{\exists}} \begin{array}{c} (\exists \exists)^\circ X \\ \hline R \end{array} \xleftrightarrow{\circ, T^\circ \cdot \neg T^\circ} \begin{array}{c} X \\ \hline \epsilon \backslash (\epsilon \backslash R) \end{array} \quad (7.3)$$

§7.1.5. $\frac{S}{\exists} \text{est}(R) = S \cap (S^\circ \backslash R^\circ)$

$$\begin{array}{c} X \\ \hline \frac{S}{\exists} \text{est}(R) \end{array} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \begin{array}{c} \frac{S}{\exists}^\circ X \\ \hline \text{est}(R) \end{array} \xleftrightarrow{\Delta \neg \cap} \begin{array}{c} \frac{S}{\exists}^\circ X \\ \hline \exists \end{array} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \begin{array}{c} X \\ \hline \frac{S}{\exists} \exists \end{array} \xleftrightarrow{\cdot \exists \neg \frac{S}{\exists}, (7.2)} \begin{array}{c} X \\ \hline S \end{array} \xleftrightarrow{\Delta \neg \cap} \begin{array}{c} X \\ \hline S \cap (S^\circ \backslash R^\circ) \end{array} \quad (7.5)$$

§7.1.6. $\frac{S}{\exists} \text{est}(R) = \frac{S}{\exists} \text{est}(R \cap S^\circ S)$

$$\begin{array}{c} X \\ \hline \frac{S}{\exists} \text{est}(R \cap S^\circ S) \end{array} \xleftrightarrow{(7.5), \circ} \begin{array}{c} X \\ \hline S \cap (S^\circ \backslash (R^\circ \cap S^\circ S)) \end{array} \xleftrightarrow{\Delta \neg \cap, T^\circ \cdot \neg T^\circ} \begin{array}{c} X \\ \hline S \end{array} \xleftrightarrow{\Delta \neg \cap, S^\circ \cdot \text{monotone}} \begin{array}{c} X \\ \hline S \end{array} \xleftrightarrow{T^\circ \cdot \neg T^\circ, \Delta \neg \cap} \begin{array}{c} X \\ \hline S \cap (S^\circ \backslash R^\circ) \end{array} \xleftrightarrow{(7.5)} \begin{array}{c} X \\ \hline \frac{S}{\exists} \text{est}(R) \end{array} \quad (7.6)$$

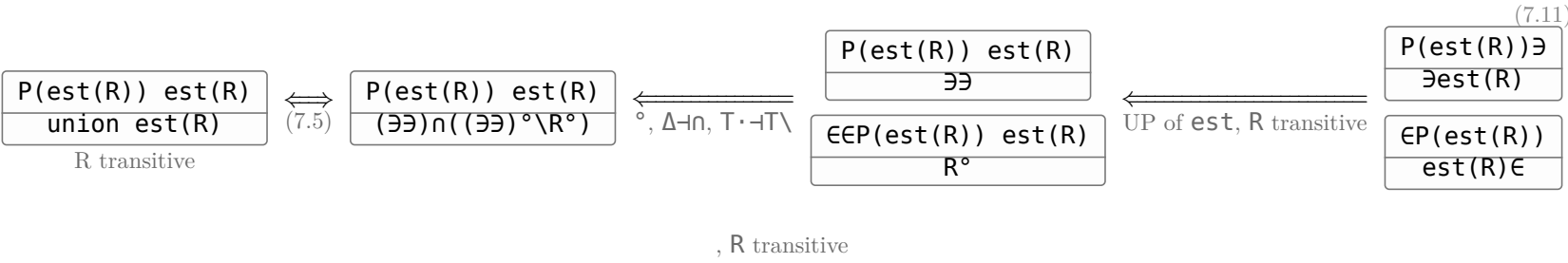
§7.1.7. **P(f)** $\text{est}(R) = \text{est}(f R f^\circ)$ **f**

$$\begin{array}{c} \text{P(f)} \text{est}(R) \\ \hline f \text{ a map} \end{array} \xrightarrow[\text{modular law, } f \text{ simple}]{\text{P=E on maps, (7.5)}} \begin{array}{c} (\exists f) \cap ((\exists f)^\circ \backslash R^\circ) \\ \hline \end{array} \xrightarrow[\cdot f \neg f^\circ, \circ, \text{est}]{\circ, T^\circ \cdot \neg T^\circ, f^\circ \cdot \neg f^\circ} \begin{array}{c} (\exists f) \cap (\epsilon \backslash (f R^\circ)) \\ \hline \end{array} \quad (7.8)$$

§7.1.8. **P(S)** $\text{est}(R) \in (\exists S) \cap (\epsilon \backslash (S R^\circ))$

$$\begin{array}{c} \text{P(S)} \text{est}(R) \\ \hline (\exists S) \cap (\epsilon \backslash (S R^\circ)) \end{array} \xleftrightarrow{\Delta \neg \cap, T^\circ \cdot \neg T^\circ} \begin{array}{c} \text{P(S)} \text{est}(R) \\ \hline \exists S \end{array} \xleftrightarrow{\text{UP of est}} \begin{array}{c} \text{P(S)} \exists \\ \hline \exists S \end{array} \quad (7.10)$$

§7.1.9. **P(est(R))** $\text{est}(R) \in \text{union est}(R)$



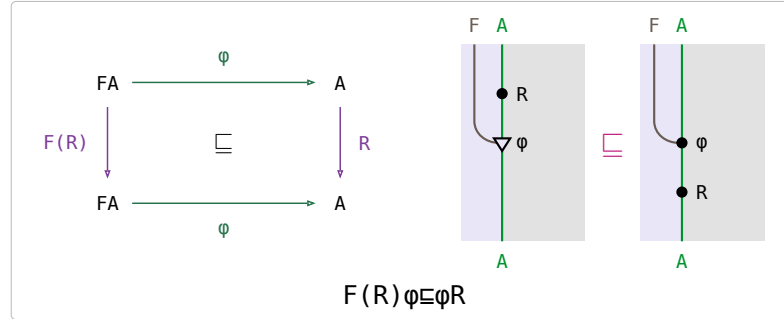
§7.2. Monotonic algebras

(7.2a)

An F -algebra $\varphi : FA \rightarrow A$ is **monotonic on** $R : A \rightarrow A$ when it is lax at R at $G := F$, $F := \text{Id}$: $F(R)\varphi \sqsubseteq \varphi R$. R is an **endorelation**, so $B=A$ and the two components φ_A, φ_B are the one arrow φ — an algebra, not a family.

For a map $f : FA \rightarrow A$ that is $f \circ F(R)f \in R$ (0.1a.15)'s $f \circ \cdot \dashv f \cdot$ at $X := F(R)f$, $Y := R$, equivalently $F(R) \sqsubseteq f R f \circ \cdot f \dashv \cdot f \circ$ then $f \circ \cdot \dashv f \cdot, .$ $(\leq \times \leq) + \sqsubseteq + \leq$ — addition on Nat is monotonic on $\leq$, which at the point level reads $c = a + b \wedge a \leq a' \wedge b \leq b' \implies c \leq a' + b'$.

(7.2b)



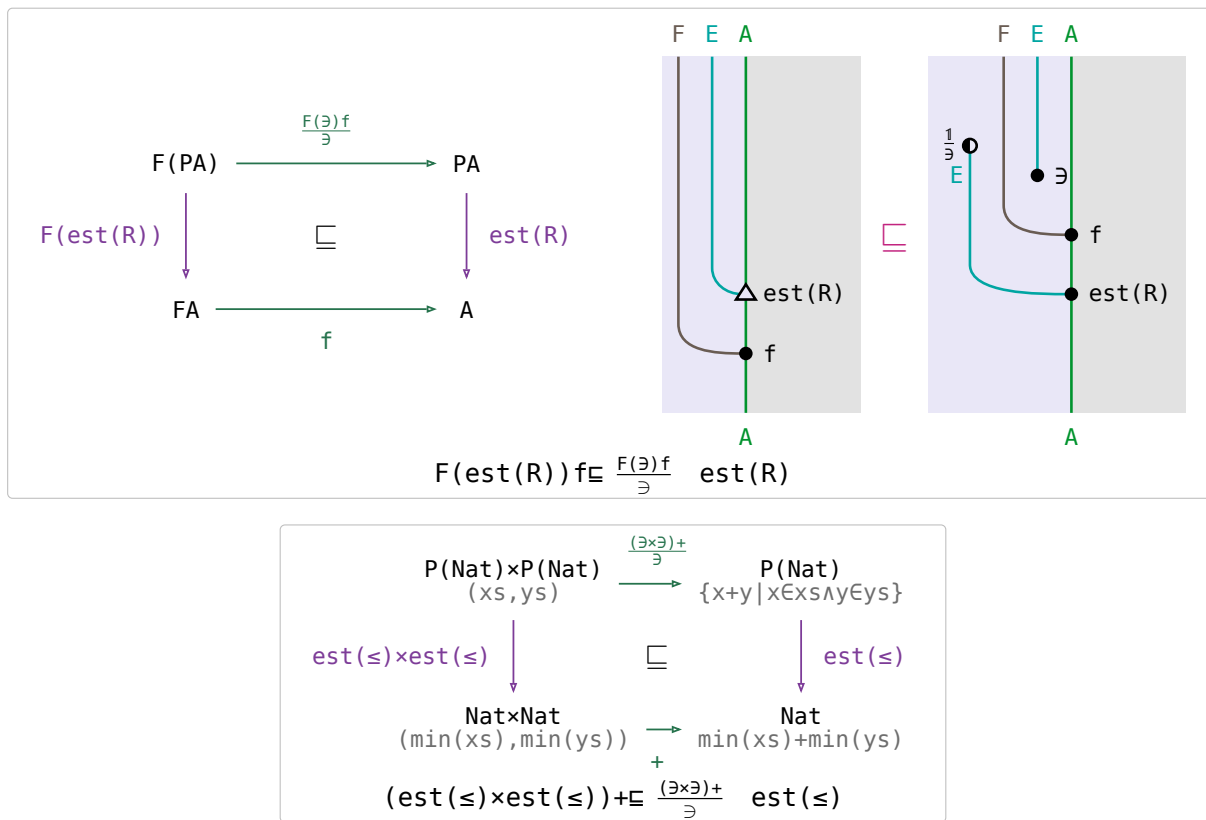
§7.2.1. Function f is monotonic on R iff it distributes over R

(7.2.1a)

$f : FA \rightarrow A$ **distributes over** R if $F(\text{est}(R))f \sqsubseteq \frac{F(\exists)f}{\exists} \text{est}(R)$.

$+$ distributes over $\leq$, at the point level $\min(xs) + \min(ys)$ is a least element of $\{x+y \mid x \in xs \wedge y \in ys\}$ for xs, ys non-empty and $\min \triangleq \text{est}(\leq)$.

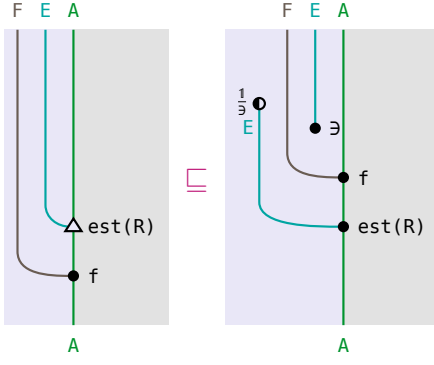
(7.2.1b)



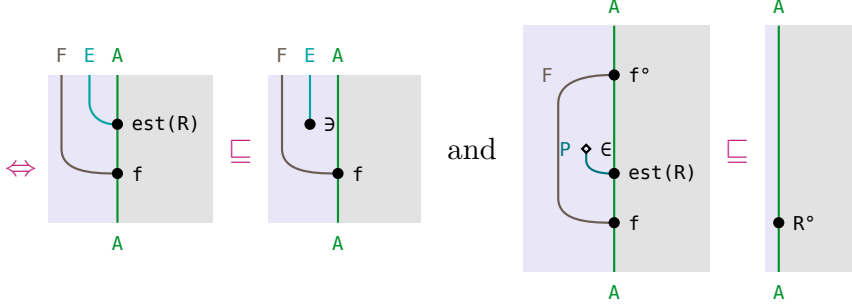
$$f^\circ F(R) f \sqsubseteq R \iff F(\text{est}(R)) f \sqsubseteq \frac{F(\exists) f}{\exists} \text{est}(R)$$

Theorem 7.1

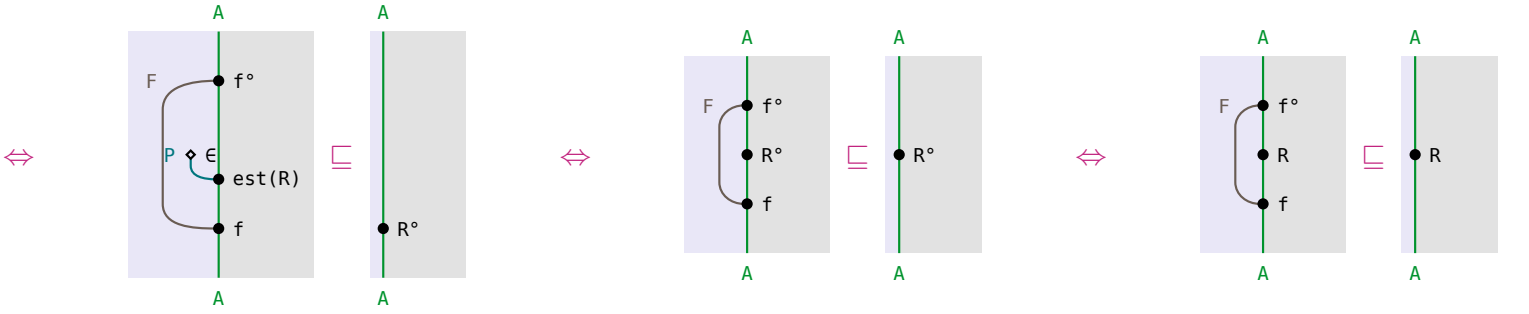
function f is monotonic over R if and only if it distributes over R ; f a map, R reflexive



f distributes over R — $\triangleq$ distributes — the fraction bent as (0.11a)



(7.5) splits the bound in two, (0.9a.2) moving $(F(\exists) f)^\circ$ across



$\text{est}(R) \sqsubseteq \exists \triangleq \text{est}$ — so the first conjunct drops

$(F(\exists) f)^\circ = f^\circ F(\epsilon) \triangleq \circ$ — and $\epsilon \text{est}(R) = R^\circ \triangleq \text{est}$, R reflexive

both sides conversed — $F(R^\circ)^\circ = F(R)$, (5.1b.3)

§7.2.3. $\text{takewhile}(p) = ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \rightarrow \text{nil})])$

(7.2.3a)

name	definition	type	example	in words
1 F	$FX = 1 + A \times X$	$\mathcal{A} \rightarrow \mathcal{A}$		nothing, or a head and a tail
2 nil, cons	$[A] ::= \text{nil} \mid \text{cons}(A, [A])$ (5.6a.1)	$1 \rightarrow [A],$ $A \times [A] \rightarrow [A]$	$\text{cons}(3, [1, 2]) = [3, 1, 2]$	the empty list; a head onto a tail
3 α	$[\text{nil}, \text{cons}]$	$F[A] \rightarrow [A]$		both constructors as one map
4 p	a coreflexive	$A \rightarrow A$	$p \not\leq \text{even} \rightarrow 2 \not\leq p \ 2$, and $3 \notin \text{dom}(p)$	$\{(a, a) \mid a \text{ passes the test}\}$
5 R	$\text{length} \leq \text{length}^\circ$, a preorder	$[A] \rightarrow [A]$	$[1] \ R \ [1, 2]$	$xs \ R \ ys \iff \text{length}(xs) \leq \text{length}(ys)$
6 $\rightarrow \text{nil}$	the constant nil — the second nil of prefix	$A \times [A] \rightarrow [A]$	$(\rightarrow \text{nil})(3, [1, 2]) = \text{nil}$	drop the pair, return nil
7 prefix	$([\text{nil}, \rightarrow \text{nil} \cup \text{cons}])$ $\text{prefix} = ([\text{nil}, \rightarrow \text{nil} \cup \text{cons}])$, $\text{prefix} = \text{cat}^\circ \pi_1$	$[A] \rightarrow [A]$	$[3, 1, 2] \ \text{prefix} \ [3, 1]$	$xs \ \text{prefix} \ ys \iff \exists zs. xs = ys \# zs$ — at each cons , stop or keep the head
8 S	$[\text{nil}, \rightarrow \text{nil} \cup (p \times 1) \text{cons}]$	$F[A] \rightarrow [A]$	$(4, [2]) \ S \ [4, 2]$, and $(3, [2]) \ S \ \text{nil}$ only	prefix 's algebra with one extra p — stop, or keep a head that passes p
9 $(g \rightarrow X, Y)$	X where g is defined and Y where it is not	$A \rightarrow B$		the test picks the branch

$\text{nil} \ R = \top$, $\text{nil} \ R^\circ = \text{nil}$ nil is the shortest list — below every list, and above only itself, so it loses every $\text{est}(R^\circ)$

$\text{prefix} = ([\text{nil}, \rightarrow \text{nil} \cup \text{cons}])$

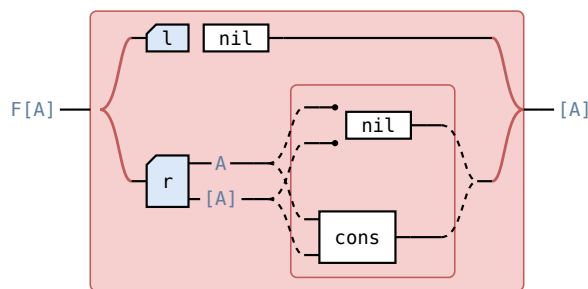
(7.2.3b)

the fold whose algebra, at each **cons**, stops with **nil** or keeps the head: $xs \ \text{prefix} \ ys \iff \exists zs. xs = ys \# zs$

the defining square $\alpha \ \text{prefix} = F(\text{prefix})[\text{nil}, \rightarrow \text{nil} \cup \text{cons}]$ — build the list and then take a prefix, or take a prefix of the tail and then rebuild with the algebra

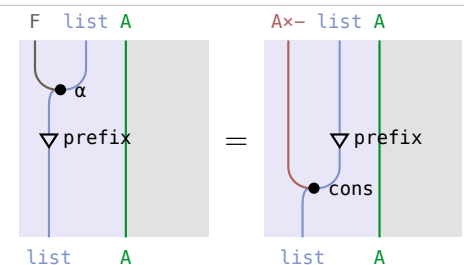


$F = 1 + A \times -$, so $F(\text{prefix}) = 1 + 1 \times \text{prefix}$: the head passes, the fold recurses on the tail alone



the algebra $[\text{nil}, \rightarrow \text{nil} \cup \text{cons}] : F([A]) \rightarrow [A]$ as a circuit: nil on the 1 branch; on a pair (a, ys') two outputs, nil and $\text{cons}(a, ys')$

Hinze–Marsden



the cons operand of $\rightarrow \text{nil} \cup \text{cons}$; $\rightarrow \text{nil}$ makes a constant and draws nothing

$\text{nil} \ \text{prefix} \ ys \iff ys = \text{nil}$
 $\text{cons}(a, xs') \ \text{prefix} \ ys \iff ys = \text{nil} \vee \exists ys'. xs' \ \text{prefix} \ ys' \wedge ys = \text{cons}(a, ys')$

a right fold: α° peels, $F(\text{prefix})$ recurses, the algebra runs on the way back; init^*
 $(\text{prefix} = ([\text{nil}, \rightarrow \text{nil} \cup \text{cons}]), \text{prefix} = \text{cat}^\circ \pi_1)$ is the tail-recursive form

lax natural only, $\text{list}(R) \ \text{prefix} \sqsubseteq \text{prefix}$
 $\text{list}(R)$: $\text{list}(R)$ first needs an R -image of every element, prefix first may have dropped the ones without

$\alpha \text{ prefix list}(p) = F(\text{prefix list}(p)) S$ (7.2.3c)	
building the list and then keeping a p -passing prefix of it is keeping one of the tail first, and then building with S this same diagram is subseq 's: algebra $[\text{nil}, \pi_2 \cup \text{cons}]$, type $[A] \rightarrow [A]$	
circuit — the fork is $F([A]) = 1 + A \times [A]$: nil above, the pair below	Hinze–Marsden
$\alpha \text{ prefix list}(p)$	the cons branch alone, without $1 +$ or $\multimap \text{nil}$
$F(\text{prefix}) [\text{nil}, \multimap \text{nil} \cup \text{cons}] \text{list}(p)$ defining equation	the cons operand of $\multimap \text{nil} \cup$ cons
$F(\text{prefix}) [\text{nil}, \multimap \text{nil} \cup (p \times \text{list}(p)) \text{cons}]$ list(p) through cons	the $(p \times \text{list}(p))$ cons operand of $\multimap \text{nil} \cup$ $(p \times \text{list}(p))$ cons
$[\text{nil}, \multimap \text{nil} \cup (p \times (\text{prefix list}(p))) \text{cons}]$ relator, prefix entire	
$F(\text{prefix list}(p)) S$ prefix list(p) entire	

$\triangleq$ **reduce** reads that off as $\text{prefix list}(p) = (S)$. fusion cannot: $\text{list}(p)$ is not entire, $(1 \times \text{list}(p)) \multimap \text{nil} \sqsubseteq \multimap \text{nil}$, and no algebra meets the side condition. ,

$(1 \times R^\circ) (\multimap \text{nil} \cup ((p \times 1) \text{cons})) \sqsubseteq (\multimap \text{nil} \cup ((p \times 1) \text{cons})) R^\circ$ (7.2.3d)	
shortening the tail and then taking the step lands inside taking the step and then shortening the result	
circuit — the cons branch of $F(R^\circ)S \in SR^\circ$	reason
$(1 \times R^\circ) (\multimap \text{nil} \cup ((p \times 1) \text{cons}))$	
$((1 \times R^\circ) \multimap \text{nil}) \cup ((p \times R^\circ) \text{cons})$	each operand is reached on its own (0.1a.9) — and $(1 \times R^\circ)$ $(p \times 1)$ is p and R° on the pair's two strands at once
$\multimap \text{nil} \cup ((p \times R^\circ) \text{cons})$	$\multimap$ is the greatest arrow into 1 , so $(1 \times R^\circ) \multimap \sqsubseteq \multimap$
$\multimap \text{nil} \cup ((p \times 1) \text{cons} R^\circ)$	$\text{cons length} = (1 \times \text{length}) \pi_2$ succ with succ monotone — a shorter tail makes a shorter list
$(\multimap \text{nil} R^\circ) \cup ((p \times 1) \text{cons} R^\circ)$	$\text{nil } R^\circ = \text{nil}$ (7.2.3a.5) — so the constant branch may carry the R° the other one already has
$(\multimap \text{nil} \cup ((p \times 1) \text{cons})) R^\circ$	one R° past the join is the two inside it (0.1a.9)

the nil branch, which no row above draws, is $\text{nil} \sqsubseteq \text{nil } R^\circ$.

$$\text{takewhile}(p) \triangleq \frac{\text{prefix list}(p)}{\exists} \text{est}(R^\circ)$$

Exercise 7.39

$$\text{takewhile}(p) = ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})])$$

takewhile: $\text{takewhile}(p)(x)$ returns the longest prefix of x with the property that all its elements satisfy p ; the catamorphism is the standard implementation.



(a)	$[\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})]$	$(7.2.3e) \frac{S}{\exists} \text{est}(R^\circ) = [\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})]$
(b)	$= ([\frac{S}{\exists} \text{est}(R^\circ)])$	$[\frac{S}{\exists} \text{est}(R^\circ)] \sqsubseteq ([\frac{(S)}{\exists} \text{est}(R^\circ)])$
(c)	$\sqsubseteq ([\frac{(S)}{\exists} \text{est}(R^\circ)])$	$\text{prefix list}(p) = (S)$
(d)	$= \frac{\text{prefix list}(p)}{\exists} \text{est}(R^\circ)$	
(e)	$= \text{takewhile}(p)$	

$$\text{takewhile}(p)^\circ \text{takewhile}(p) \sqsubseteq \text{prefix}^\circ \text{prefix} \cap R \cap R^\circ \sqsubseteq 1$$

$\text{takewhile}(p) \sqsubseteq \text{prefix list}(p)$ and $(\text{prefix list}(p))^\circ \text{takewhile}(p) \in R$ — (7.5) at $\text{est}(R^\circ)$ — and two prefixes of one list of equal length are equal, so $\text{takewhile}(p)$ is simple: the longest, not a longest.

$$\frac{\text{prefix list}(p)}{\exists} \text{est}(R^\circ) \text{ entire}$$

nil is always a p -prefix and R is connected on the prefixes of one list, so the longest exists.

$$X \sqsubseteq Y, X \text{ entire}, Y \text{ simple} \implies X = Y$$

$[\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})]$ is a reduce of maps, hence entire — what turns the $\sqsubseteq$ above into the heading's $=$.

$$\S 7.2.4. \text{mss} = ([\text{zero}(1, \frac{1}{\exists}), ((1 \times \pi_1) \oplus, ((1 \times \pi_1) \oplus \frac{1}{\exists}, \pi_2 \pi_2) \text{cup})]) \pi_2 \text{est}(\geq)$$

(7.2.4a)

$FX = 1 + A \times X$, $\alpha \triangleq [\text{nil}, \text{cons}]$, $\text{sum} = ([\text{zero}, \text{plus}])$ and $\text{segment} = \text{suffix prefix}$ from $\text{sum} = ([\text{zero}, \text{plus}])$ and $\text{segment} = \text{suffix prefix}$.

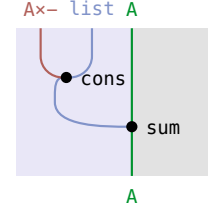
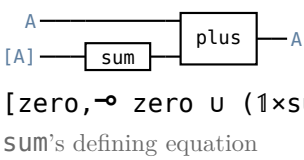
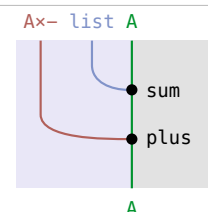
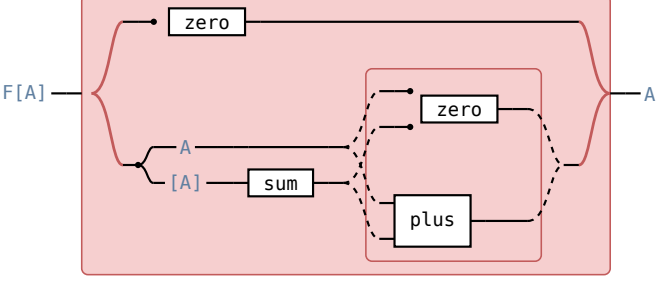
$\text{head} \triangleq \text{cons}^\circ \pi_1$, $\text{wrap} \triangleq (1, \neg \text{nil}) \text{cons}$ the head of a list and the one-element list, beside $\text{suffix} = \text{cat}^\circ \pi_2$'s $\text{tail} \triangleq \text{cons}^\circ \pi_2$

$\oplus \triangleq \frac{\neg \text{zero} \cup \text{plus}}{\exists} \text{est}(\geq)$ the set at (a, b) is $\{0, a+b\}$, so $\oplus$ is the larger of the two,

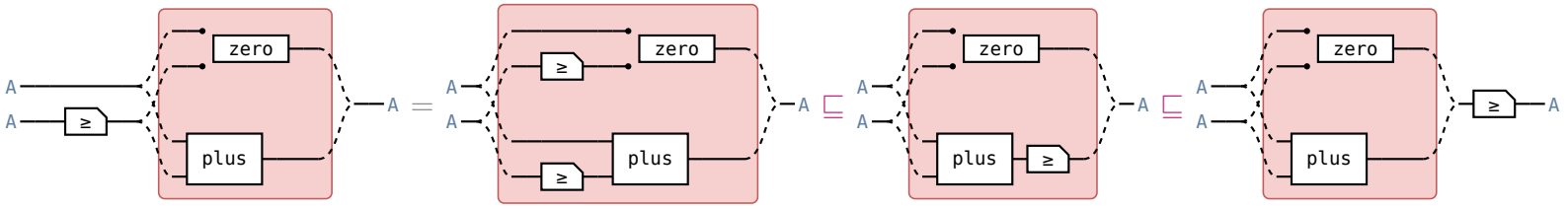
$\text{mss} = \frac{\text{suffix}}{\exists} E\left(\frac{\text{prefix sum}}{\exists} \text{est}(\geq)\right) \text{est}(\geq) \quad (7.2.4b)$	
maximum segment sum problem: using segment=suffix prefix , the specification is expressed in this form	
formula — one wire from [A] to A, its type written along it	reason
$[A] \xrightarrow{\frac{1}{\exists}} P[A] \xrightarrow{E(\text{segment sum})} PA \xrightarrow{\text{est}(\geq)} A$	
$= [A] \xrightarrow{\frac{1}{\exists}} P[A] \xrightarrow{E(\text{suffix prefix sum})} PA \xrightarrow{\text{est}(\geq)} A$	segment=suffix prefix segment = suffix prefix, $\triangleq$ mss
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) P[A] \xrightarrow{E(\text{prefix sum})} PA \xrightarrow{\text{est}(\geq)} A$	absorption $\frac{S}{\exists} E(R) = \frac{S}{\exists} \frac{R}{\exists}$ — $\text{frac}(S, \exists)$ $E(R) = \text{frac}(SR, \exists)$ at $S := \text{suffix}, R := \text{prefix}$ sum
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) P[A] \xrightarrow{E\left(\frac{\text{prefix sum}}{\exists}\right)} PPA \xrightarrow{\text{union}} PA \xrightarrow{\text{est}(\geq)} A$	$\frac{R}{\exists} \exists = R, \text{union} = E(\exists)$ $\frac{R}{\exists} \exists = R$ $\text{frac}(R, \exists) \exists = R$ at $R := \text{prefix sum}$ and $E(R) \triangleq \text{frac}(\exists R, \exists)$, $E(R) \triangleq \frac{\exists R}{\exists}$; (7.1b.4) $\text{union} \triangleq \text{frac}(\exists \exists, \exists)$; the middle equality is $\triangleq$ relator's $F(RS) = F(R)F(S)$ at $F := E$
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) P[A] \xrightarrow{E\left(\frac{\text{prefix sum}}{\exists}\right)} PPA \xrightarrow{E(\text{est}(\geq))} PA \xrightarrow{\text{est}(\geq)} A$	$R R \subseteq R$ $\Rightarrow P(\text{est}(R)) \text{est}(R)$ $= P(\text{dom}(\text{est}(R)))$ union $\text{est}(R)$, the sets non-empty
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) P[A] \xrightarrow{E\left(\frac{\text{prefix sum}}{\exists} \text{est}(\geq)\right)} PA \xrightarrow{\text{est}(\geq)} A$	relator $\triangleq$ relator — $F(RS) = F(R)F(S)$ at $F := E$

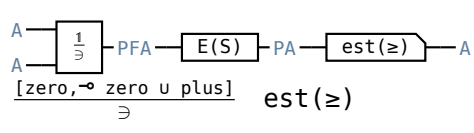
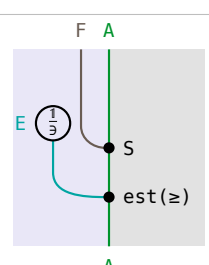
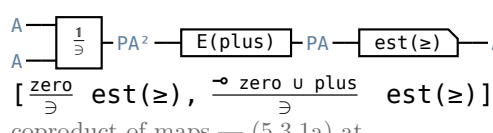
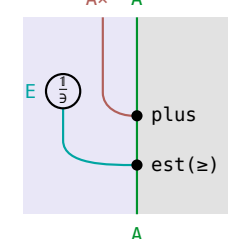
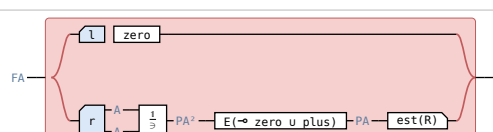
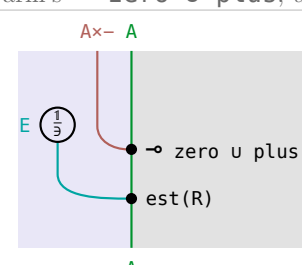
The union step is $R R \subseteq R$

$\Rightarrow P(\text{est}(R)) \text{est}(R) = P(\text{dom}(\text{est}(R))) \text{union est}(R)$'s $P(\text{est}(R)) \text{est}(R) = P(\text{dom}(\text{est}(R))) \text{union est}(R)$ — every suffix has the empty prefix, so Dom is 1 here — and $P(f) = E(f)$ at the map it is applied to (Map(f))
 $\Rightarrow P(f) = E(f)$.

$[nil, \multimap nil \cup cons] \text{ sum} = F(\text{sum}) \text{ S}$ (7.2.4c)	
fusion: prefix expressed as a catamorphism on cons-lists, $([nil, \multimap nil \cup cons])$, and this is the fusion condition that expresses prefix sum as a catamorphism	
circuit — the fork is the bracket's case split $F([A])=1+A \times [A]$: nil above, the pair and its u below	Hinze–Marsden
 $[nil, \multimap nil \cup cons] \text{ sum}$	 the cons operand of $\multimap nil \cup cons$
 $[nil \text{ sum}, \multimap nil \text{ sum} \cup cons \text{ sum}]$ coproduct of maps, composition over u	
 $[zero, \multimap zero \cup (1 \times sum) \text{ plus}]$ sum's defining equation	 the $(1 \times sum)$ plus operand of $\multimap zero \cup (1 \times sum)$ plus
 $[zero, (1 \times sum) (\multimap zero \cup plus)]$ $(1 \times sum) \multimap = \multimap$, sum entire	
 $F(\text{sum}) [zero, \multimap zero \cup plus]$ relator	

fusion at $R := [nil, \multimap nil \cup cons]$, $S := \text{sum}$: the side condition, so **prefix sum** $= ([zero, \multimap zero \cup plus])$. **prefix** is the reduce, **sum** the map fused into it — the intermediate list is gone.

MonoAlg(S, ≥) (7.2.4d) monotonic algebra: an F -algebra S is monotonic on a relation R if $F(R) \text{SES} R$ — the plus branch of $F(\geq) \text{SES} \geq$; the zero branch is $\text{zero} \text{E} \text{zero} \geq$	
circuit — the head above, the running sum below; the tape is the u $(1 \times \geq) (\neg \text{zero} \cup \text{plus}) \quad ((1 \times \geq) \neg \text{zero}) \cup ((1 \times \neg \text{zero} \cup (\text{plus} \geq) \neg \text{zero} \cup \text{plus}) \geq$  relator, composition over U $(0.8a), (\geq \times \geq) \text{plus} \text{E} \text{plus} \geq; \geq \text{ reflexive}$ $(\leq \times \leq) \text{plus} \text{E} \text{plus} \leq \text{ is } \triangleq$ monotonic, written + there, and plus is a map, so it is monotonic on an order and on its opposite together, which carries it to $\geq$.	

$\frac{S}{\geq} \text{est}(\geq) = [\text{zero}, \oplus]$ (7.2.4e) the largest sum the algebra offers is zero from nothing and, from a head and a running sum, the larger of zero and the head added to it	
circuit — the tape is the coproduct: zero's branch above, plus's below 	Hinze–Marsden 
 coproduct of maps — (5.3.1a) at $T := [\text{zero}, \neg \text{zero} \cup \text{plus}]$, then $[U, V]Z = [UZ, VZ] \text{ — } [R, S] Z = [RZ, SZ]$, composition over U	
 singleton, $\geq$ reflexive — (7.1b.6)'s $\frac{1}{\geq}$ $\text{est}(R) = 1 \text{ n} R$ at $R := \geq$, zero a map; the lower branch is $\oplus$'s definition, $\triangleq \text{mss}$, and no law	

with (7.2.4d) **MonoAlg(S, ≥)** the greedy theorem gives $([\text{zero}, \oplus]) \text{E} \frac{\text{prefix sum}}{\geq} \text{est}(\geq)$ — and Exercise 7.40 $\text{mss} = (k) \pi_2 \text{est}(\geq)$ makes it an equality.

$$[\text{nil}, \text{cons}] (\llbracket [\text{zero}, \oplus] \rrbracket, \frac{\text{suffix}}{\exists} E(\llbracket [\text{zero}, \oplus] \rrbracket)) = F(\llbracket [\text{zero}, \oplus] \rrbracket, \frac{\text{suffix}}{\exists} E(\llbracket [\text{zero}, \oplus] \rrbracket)) k \quad (7.2.4f)$$

$k \triangleq [\text{zero}(\frac{1}{\exists}), \langle w, \langle w \frac{1}{\exists}, \pi_2 \pi_2 \rangle \text{cup} \rangle]$, $w \triangleq (1 \times \pi_1) \oplus$: the value at the whole list, paired with the set of the values at its suffixes, runs k 's recursion.

the equation at that branch

$$\text{nil}(g, \frac{\text{suffix}}{\exists} E(g)) = \text{zero}(\frac{1}{\exists})$$

$$\text{cons}(g, \frac{\text{suffix}}{\exists} E(g)) = (1 \times \langle g, \frac{\text{suffix}}{\exists} E(g) \rangle) \langle w, \langle w \frac{1}{\exists}, \pi_2 \pi_2 \rangle \text{cup} \rangle$$

why

nil has one suffix, itself, and $g(\text{nil}) = \text{zero}$, so the set is the singleton $\{\text{zero}\}$

$g(\text{cons}(a, x)) = a \oplus (g(x))$, which is w reading $g(x)$ off π_1 ; and the suffixes of $\text{cons}(a, x)$ are $\text{cons}(a, x)$ itself, whose value is that same w , together with those of x , which $\pi_2 \pi_2$ carries — so the two sets meet at cup

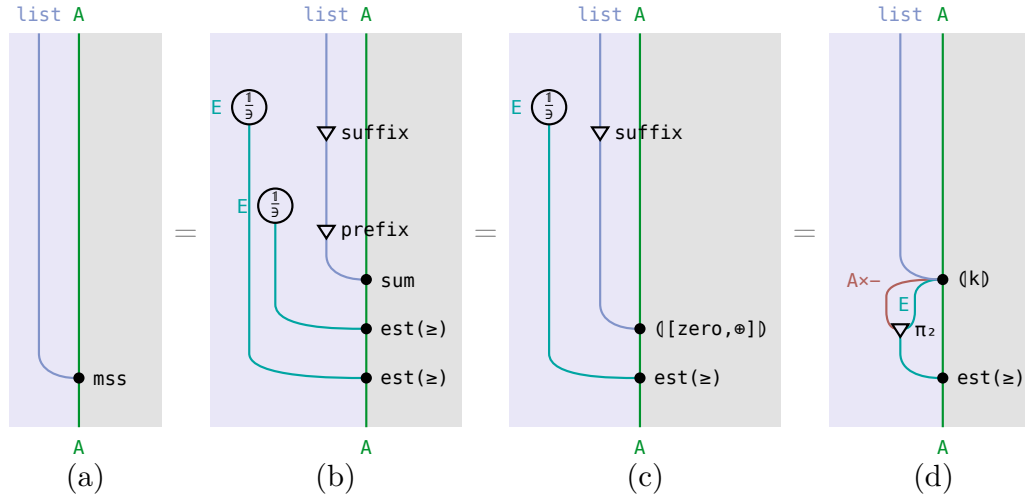
$$\text{mss} = (\llbracket k \rrbracket \pi_2 \text{est}(\geq))$$

Exercise 7.40

maximum segment sum problem: $\frac{\text{suffix}}{\exists} E(\llbracket [\text{zero}, \oplus] \rrbracket)$ expressed as the catamorphism $(\llbracket k \rrbracket)$, hence

mss implemented by a linear-time algorithm, $\oplus \triangleq \frac{\text{zero} \vee \text{plus}}{\exists} \text{est}(\geq) \triangleq \text{mss}$; k and w — (7.2.4f)

$$[\text{nil}, \text{cons}] (\llbracket [\text{zero}, \oplus] \rrbracket, \frac{\text{suffix}}{\exists} E(\llbracket [\text{zero}, \oplus] \rrbracket)) = F(\llbracket [\text{zero}, \oplus] \rrbracket, \frac{\text{suffix}}{\exists} E(\llbracket [\text{zero}, \oplus] \rrbracket)) k.$$



(a)	mss	$(7.2.4b) \text{mss} = \frac{\text{suffix}}{\exists} E(\frac{\text{prefix sum}}{\exists} \text{est}(\geq)) \text{est}(\geq)$
(b)	$= \frac{\text{suffix}}{\exists} E(\frac{\text{prefix sum}}{\exists} \text{est}(\geq)) \text{est}(\geq)$	$\frac{\text{prefix sum}}{\exists} \text{est}(\geq) = \llbracket [\text{zero}, \oplus] \rrbracket$
(c)	$= \frac{\text{suffix}}{\exists} E(\llbracket [\text{zero}, \oplus] \rrbracket) \text{est}(\geq)$	$(\llbracket k \rrbracket \pi_2 = \frac{\text{suffix}}{\exists} E(\llbracket [\text{zero}, \oplus] \rrbracket))$
(d)	$= (\llbracket k \rrbracket \pi_2 \text{est}(\geq))$	

one fold builds the $n+1$ running maxima and the final $\text{est}(\geq)$ reads them in one more pass, so mss is linear.

§7.2.5. $\text{filter}(p) = ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \pi_2)])$

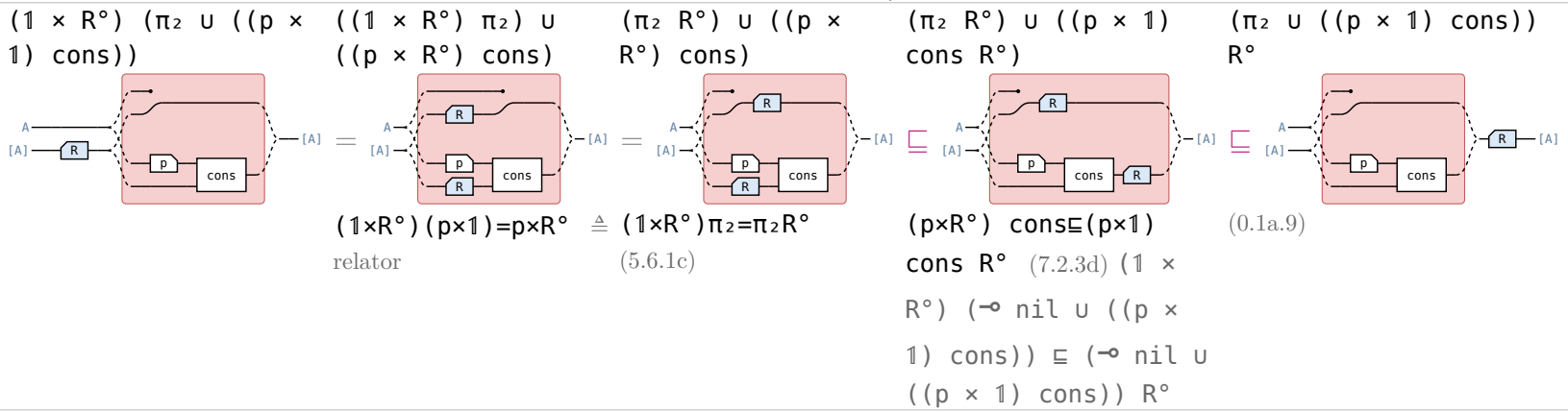
(7.2.5a)

name	definition	type	example	in words
1 F, α, p, R	as in (7.2.3a.1), (7.2.3a.3), (7.2.3a.4), (7.2.3a.5)			
2 π_2	π_2 where (7.2.3a.6) has $\rightarrow \text{nil}$	$A \times [A] \rightarrow [A]$	$\pi_2(3, [1, 2]) = [1, 2]$	drop the head, keep the tail
3 subseq	$([\text{nil}, \pi_2 \cup \text{cons}])$ $\text{subseq} = ([\text{nil}, \text{cons} \cup \pi_2])$	$[A] \rightarrow [A]$	$[3, 1, 2] \text{ subseq } [3, 2]$	$xs \text{ subseq } ys \iff ys \text{ is } xs \text{ with elements dropped}$ — at each cons , drop the head or keep it
4 S	$[\text{nil}, \pi_2 \cup (p \times 1) \text{ cons}]$	$F[A] \rightarrow [A]$	$(4, [2]) S [2]$ and $(4, [2]) S [4, 2]$, but $(3, [2]) S [2]$ only	subseq 's algebra with one extra p — drop the head, or keep a head that passes p
5 $1 \sqsubseteq \pi_2 R \text{ cons}^\circ$				the tail is one shorter than the cons, so π_2 loses the $\text{est}(R^\circ)$ at every step — where (7.2.3a.6)'s loser is nil

MonoAlg(S, R°)

(7.2.5b)

shortening the tail and then taking the step lands inside taking the step and then shortening the result

formula — the cons branch of $F(R^\circ) S \sqsubseteq SR^\circ$; **reason** under each circuitthe nil branch: $\text{nil} \sqsubseteq \text{nil } R^\circ$.

$\frac{S}{\text{est}} \text{est}(R^\circ) = [\text{nil}, (\pi_1 p \rightarrow \text{cons}, \pi_2)]$ (7.2.5c)	
the longest of the lists the algebra allows is the cons where the head passes p , and the tail where it does not	
formula	reason
	$S = [\text{nil}, \pi_2 \cup (p \times 1) \text{ cons}]$ (7.2.5a.4) — and the $\frac{\%}{3}$ of a coproduct of maps is the coproduct of their $\frac{\%}{3}$ (5.3.1a)
	$\text{nil} \frac{\%}{3}$ is the singleton $\{\text{nil}\}$, and $\text{est}(R^\circ)$ of a singleton is its element because R° is reflexive $\triangleq \text{est}$
	$\frac{\pi_2 \cup (p \times 1) \text{ cons}}{\frac{\%}{3}} = \left(\frac{\pi_2}{\frac{\%}{3}}, \frac{(p \times 1) \text{ cons}}{\frac{\%}{3}} \right) \text{ cup} \triangleq \text{cup}$
	$1 \sqsubseteq \pi_2 R \text{ cons}^\circ$ (7.2.5a.5) — $x s \ R \text{ cons}(a, xs)$, so $\text{est}(R^\circ)$ returns the cons where p a puts it in the set and xs where the set is $\{xs\} \triangleq \text{est}$

the head is dropped, not the whole tail: that is the one place π_2 shows against (7.2.3e) $\frac{S}{\text{est}} \text{est}(R^\circ) = [\text{nil}, (\pi_1 p \rightarrow \text{cons}, \rightarrow \text{nil})]$'s $\rightarrow \text{nil}$.

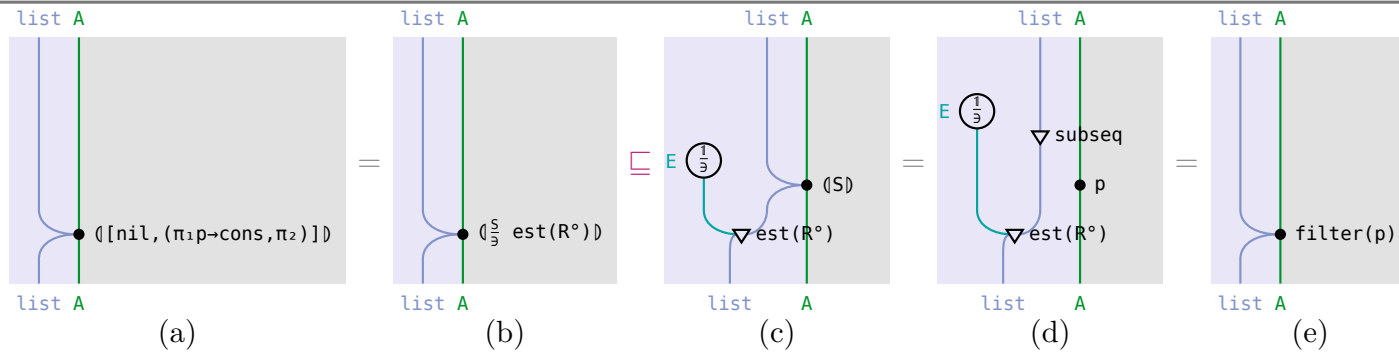
(7.2.5d)

name	definition	type	example	in words
S	$[\text{nil}, \pi_2 \cup (p \times 1) \text{ cons}]$ (7.2.5a.4)	$F[A] \rightarrow [A]$	$(4, [2]) \ S \ [2]$ and $(4, [2]) \ S \ [4, 2]$, but $(3, [2]) \ S \ [2]$ only	subseq's algebra with one extra p — drop the head, or keep a head that passes p

$$\mathbf{filter}(p) = ([\mathbf{nil}, (\pi_1 p \rightarrow \mathbf{cons}, \pi_2)])$$

Exercise 7.41

filter: $\mathbf{filter}(p)$ x returns the longest subsequence of x with the property that all its elements satisfy p ; the catamorphism is the standard program; R a preorder.



(a)	$[\mathbf{nil}, (\pi_1 p \rightarrow \mathbf{cons}, \pi_2)]$	$(7.2.5c) \frac{S}{\exists} \mathbf{est}(R^\circ) = [\mathbf{nil}, (\pi_1 p \rightarrow \mathbf{cons}, \pi_2)]$
(b)	$= [\frac{S}{\exists} \mathbf{est}(R^\circ)]$	$[\frac{S}{\exists} \mathbf{est}(R^\circ)] \sqsubseteq [\frac{S \mathbf{D}}{\exists} \mathbf{est}(R^\circ)]$
(c)	$\sqsubseteq [\frac{S \mathbf{D}}{\exists} \mathbf{est}(R^\circ)]$	$\mathbf{subseq} \mathbf{list}(p) = \mathbf{D} \mathbf{S}$
(d)	$= \frac{\mathbf{subseq} \mathbf{list}(p)}{\exists} \mathbf{est}(R^\circ)$	
(e)	$= \mathbf{filter}(p)$	

the catamorphism is entire and $\mathbf{filter}(p)$ simple, so $\sqsubseteq$ is $=$.

$(\mathbf{subseq} \mathbf{list}(p))^\circ (\mathbf{subseq} \mathbf{list}(p)) \cap R \cap R^\circ \not\sqsubseteq 1$ fails — two p -subsequences of one list can be of equal length and different — so §Section 7.2.3's uniqueness argument does not transfer. What survives $\mathbf{est}(R^\circ)$ is the one keeping **exactly** the passing elements: a subsequence that drops a passing element is beaten by the one that keeps it.

(7.2.5e)

	definition	type	note
1	segment = suffix prefix	$[A] \rightarrow [A]$	A contiguous stretch of x s: a suffix, then a prefix of that.
2	$\mathbf{filter}(p) \triangleq \frac{\mathbf{subseq} \mathbf{list}(p)}{\exists} \mathbf{est}(R^\circ)$	$[A] \rightarrow [A]$	The longest subsequence of x s whose every element passes p . $\mathbf{est}(R^\circ)$ is $\triangleq \mathbf{est}$
3	$R \triangleq \mathbf{length} \leq \mathbf{length}^\circ$	$[A] \rightarrow [A]$	The preorder filter and takewhile maximise over: the longer list wins. $\geq \triangleq \leq^\circ$
4	$\mathbf{takewhile}(p) \triangleq \frac{\mathbf{prefix} \mathbf{list}(p)}{\exists} \mathbf{est}(R^\circ)$	$[A] \rightarrow [A]$	The same with prefix for subseq : the longest prefix whose every element passes p .
5	$\mathbf{mss} \triangleq \frac{\mathbf{segment} \mathbf{sum}}{\exists} \mathbf{est}(\geq)$	$[A] \rightarrow A$	Maximum segment sum. segment=suffix prefix splits it into $\frac{\mathbf{prefix} \mathbf{sum}}{\exists} \mathbf{est}(\geq)$ on each suffix.

§7.3. Planning a company party

(7.3a)

	definition	type	note
1	$\text{tree } A ::= \text{node } (A, [\text{tree } A])$	$\mathcal{A} \rightarrow \mathcal{A}$	The company hierarchy: an employee, and the list of subtrees under them.
2	$F(A, B) = A \times [B]$	$\mathcal{A} \rightarrow \mathcal{A}$	The base functor tree folds: an employee beside the recursive position, one layer deep.
3	rating	$A \rightarrow \text{Int}$	What one employee is worth as a guest.
4	$\text{cost} = \text{list}(\text{rating}) \text{ sum}$	$[A] \rightarrow \text{Int}$	What a guest list is worth.
5	$R = \text{cost} \leq \text{cost}^\circ$	$[A] \rightarrow [A]$	The preorder the guest list is maximised over.
6	$\text{choose} \triangleq \pi_1 \cup \pi_2$	$[A] \times [A] \rightarrow [A]$	Takes one of the two parties a subtree returns.
7	$\text{include} = (1 \times (\text{list}(\pi_2) \text{ concat})) \text{ cons}$	$F(A, [A] \times [A]) \rightarrow [A]$	The party that invites the root, which puts every immediate subtree's root out. A map.
8	$\text{exclude} = (1 \times (\text{list}(\text{choose}) \text{ concat})) \pi_2$	$F(A, [A] \times [A]) \rightarrow [A]$	The party that leaves the root out, so each subtree is free to choose. Not a map.
9	$S \triangleq (\text{include}, \text{exclude})$	$F(A, [A] \times [A]) \rightarrow [A] \times [A]$	The algebra: one step returns both parties of a subtree at once.
10	$\text{party} = (S) \text{ choose}$	$\text{tree}(A) \rightarrow [A]$	Every guest list the president's ruling allows.
11	the specification $\frac{\text{party}}{\exists} \text{ est}(R^\circ)$	$\text{TA} \rightarrow [A]$	A guest list of greatest total conviviality.

§7.3.1. include and exclude

(7.3.1a)



the small grey number is the employee's rating

(7.3.1b)

	node	the algebra's input $A \times [[A] \times [A]]$	include	exclude
1	d, e, f	$(d, [])$	$[d]$	$[]$
2	b	$(b, ([[d], []], ([e], []]))$	$[b]$	$[d, e], [d], [e], []$
3	c	$(c, ([[f], []]))$	$[c]$	$[f], []$
4	a	$(a, ([[b], ...], ([c], ...)))$	$[a] \# \text{exclude}(b) \# \text{exclude}(c)$	$\text{choose}(b) \# \text{choose}(c)$

with those ratings, $\text{est}(R^\circ)$ keeps $([b]=7, [d, e]=6)$ at **b**, where **include** wins, and $([c]=2, [f]=8)$ at **c**, where **exclude** wins, so at the root **include**=[a,d,e,f]=17 beats **exclude**=[b,f]=15, and **choose** takes 17.

(7.3.1c)



$$A \xrightarrow{\quad} \text{list}(\text{choose}) \xrightarrow{\quad} \text{concat} \xrightarrow{\quad} A$$

(7.3.1d)

list(choose) [([d],[]), ([e],[])]
 1st item ([d],[]) choose $\rightarrow$ [d] or []
 2nd item ([e],[]) choose $\rightarrow$ [e] or []
 2x2=4 combinations: [[d],[e]] [[d],[]] [[],[e]] [[],[]]
 concat flattens: [d,e] [d] [e] []

(7.3.1e)

§7.3.2. list((R×R)^o)

$$\begin{array}{c} [([b],[d,e]) , ([c],[f])] \\ \downarrow \quad \downarrow \\ [([b],[d]) , ([c],[])] \end{array} \quad \text{list}((R \times R)^o), \text{elementwise}$$

(7.3.2a)

([b],[d,e])
 | — exclude: the best party in b's subtree when b does not come
 | — include: the best party in b's subtree when b comes

(7.3.2b)

element	1st component include	2nd component exclude
1 1st, ([b],[d,e])	[b] R ^o [b] 7 ≥ 7, reflexivity	[d,e] R ^o [d] 6 ≥ 5, and ([b],[d,e]) is the pair est(R ^o) keeps at b
2 2nd, ([c],[f])	[c] R ^o [c] 2 ≥ 2, reflexivity	[f] R ^o [] 8 ≥ 0, and ([c],[f]) is the pair est(R ^o) keeps at c

the first component of $1 \times \text{list}((R \times R)^o)$ is the root employee.

$R \triangleq \text{cost} \leq \text{cost}^o$ is a preorder, so the components that do not move are related by reflexivity — $(R \times R)^o$ demands a relation in **both** components, it cannot skip one.

R compares cost only, not contents: [f] R^o [] is legal though [f] has an element [] does not. That is why the three leaves of §Section 7.3.3's proof all reduce to "cost is a sum".

§7.3.3. (1×list((R×R)^o))S⊆S(R×R)^o — S : F([A]×[A])→[A]×[A] monotonic on (R×R)^o

(7.3.3a)



(7.3.3b)

$$\begin{array}{ccc} \text{tree}(A) & \xrightarrow{\text{S} \text{ choose}} & P[A] \\ \text{tree}(A) & \xrightarrow{\text{S}} & P([A] \times [A]) \xrightarrow{E(\text{choose})} P[A] \\ \text{S} \text{ choose} = \text{S} & E(\text{choose}) & \text{S} E(R) = \text{S} R, \text{absorption,} \end{array}$$

$$\begin{aligned}
S &\triangleq \{\text{include}, \text{exclude}\} \\
(1 \times \text{list}((R \times R)^\circ)) S &\sqsubseteq S (R \times R)^\circ \\
(1 \times \text{list}((R \times R)^\circ)) \text{include} &\sqsubseteq \text{include } R^\circ \\
(1 \times \text{list}((R \times R)^\circ)) \text{exclude} &\sqsubseteq \text{exclude } R^\circ
\end{aligned}
\tag{7.3.3c}$$

bettering both parties of every subtree before the node's algebra runs gets no further than running it first and bettering the two parties it returns,



(a)	$(1 \times \text{list}((R \times R)^\circ)) (1 \times (\text{list}(g) \text{ concat})) h$	
(b)	$\sqsubseteq (1 \times (\text{list}(g R^\circ) \text{ concat})) h$	
(c)	$\sqsubseteq (1 \times (\text{list}(g) \text{ concat } R^\circ)) h$	
(d)	$\sqsubseteq (1 \times (\text{list}(g) \text{ concat})) h R^\circ$	

$g \quad (R \times R)^\circ g \sqsubseteq g R^\circ$
 $g := \pi_2$ is $(R \times R)^\circ \pi_2 = (\text{dom}(\pi_1 R^\circ)) \pi_2 R^\circ \sqsubseteq \pi_2 R^\circ$, $g := \pi_1$ its mirror $(\text{dom}(\pi_2 R^\circ)) \pi_1 R^\circ \sqsubseteq \pi_1 R^\circ \triangleq \times$ and (5.2a), then $\text{dom} \sqsubseteq 1$;
 $g := \text{choose} \triangleq \pi_1 \cup \pi_2$ is the union of the two $G(R) \ (\varphi_B \cup \psi_B) \sqsubseteq (\varphi_A \cup \psi_A) F(R)$. list monotonic, $\triangleq$ relator
 $\text{concat} \quad \text{list}(R^\circ) \text{ concat} \sqsubseteq \text{concat } R^\circ$
a LEAF: no law above it. cost is a sum, so $\text{cost}(\text{concat}(xss)) = \text{sum}(\text{list}(\text{cost})(xss))$ and a cheaper part makes a cheaper whole.
 $h \quad (1 \times R^\circ) h \sqsubseteq h R^\circ$
a LEAF: $\text{cost}(\text{cons}(a, xs)) = \text{rating}(a) + \text{cost}(xs)$, so a cheaper tail makes a cheaper list. For $h := \pi_2$ it is an EQUALITY
 $(1 \times R^\circ) \pi_2 = (\text{dom}(\pi_1)) \pi_2 R^\circ = \pi_2 R^\circ : \pi_1 \text{ is a map, hence entire, so } \text{dom}(\pi_1) = 1 \text{ — Entire}(R) \triangleq \text{dom}(R) = 1$

$(1 \times (\text{list}(g) \text{ concat})) h$	g	h
include	π_2	cons
exclude	choose	π_2

(7.3.3d)

1	$R \triangleq \{(0,0)\} : \{0,1\} \rightarrow \{0,1\}$	not entire — undefined at 1
2	$(R \times R) \text{ choose at } (0,1)$	nothing — $R \times R$ needs BOTH components related, and R is undefined at 1
3	$\text{choose } R \text{ at } (0,1)$	0 — choose picks the first component 0, and $R \ 0 \ 0$ holds
4	$((0,1), 0)$	in choose R , not in $(R \times R) \text{ choose}$
5	$(R \times R) \text{ choose} = \text{choose } R$	iff R is entire — $\text{dom} \sqsubseteq 1$ is the g row's only inequality step, and it is an equality iff $\text{dom}(R) = 1$

§7.3.4. The derivation

$$(\langle \text{include}, \pi_2 \text{ list}(\frac{\text{choose}}{3} \text{ est}(R^\circ)) \text{ concat} \rangle) \frac{\text{choose}}{3} \text{ est}(R^\circ) \sqsubseteq \frac{\text{party}}{3} \text{ est}(R^\circ) \quad (7.3.4a)$$

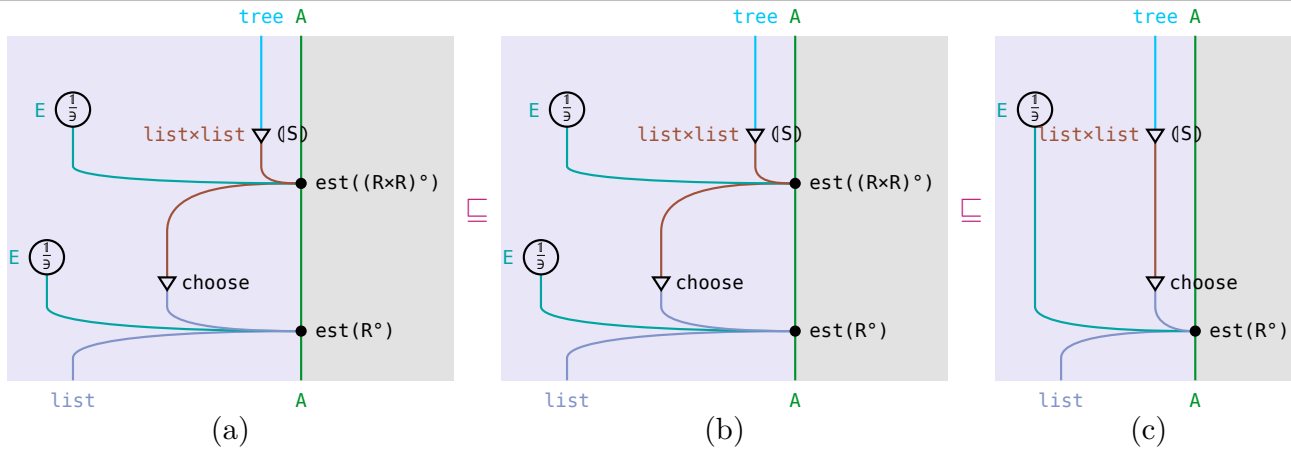
the best of every guest list the president allows is one pass up the tree, each subtree handing up its best party with its boss in and its best with the boss out, and **choose** taking the better of the two at the root



(a)	$(\langle \text{include}, \pi_2 \text{ list}(\frac{\text{choose}}{3} \text{ est}(R^\circ)) \text{ concat} \rangle) \frac{\text{choose}}{3} \text{ est}(R^\circ)$	$\pi_2 \text{ list}(\frac{\text{choose}}{3} \text{ est}(R^\circ)) \text{ concat} \sqsubseteq \frac{\text{exclude}}{3} \text{ est}(R^\circ)$
(b)	$\sqsubseteq (\langle \text{include}, \frac{\text{exclude}}{3} \text{ est}(R^\circ) \rangle) \frac{\text{choose}}{3} \text{ est}(R^\circ)$	$\text{include} \sqsubseteq \frac{\text{include}}{3} \text{ est}(R^\circ)$
(c)	$\sqsubseteq (\langle \frac{\text{include}}{3} \text{ est}(R^\circ), \frac{\text{exclude}}{3} \text{ est}(R^\circ) \rangle) \frac{\text{choose}}{3} \text{ est}(R^\circ)$	$(\frac{\text{include}}{3} \text{ est}(R^\circ), \frac{\text{exclude}}{3} \text{ est}(R^\circ)) \sqsubseteq \frac{S}{3} \text{ est}((R \times R)^\circ)$
(d)	$\sqsubseteq (\frac{S}{3} \text{ est}((R \times R)^\circ)) \frac{\text{choose}}{3} \text{ est}(R^\circ)$	$(\frac{S}{3} \text{ est}((R \times R)^\circ)) \sqsubseteq (\frac{QS}{3} \text{ est}((R \times R)^\circ))$
(e)	$\sqsubseteq (\frac{QS}{3} \text{ est}((R \times R)^\circ)) \frac{\text{choose}}{3} \text{ est}(R^\circ)$	

(7.3.4b)

1



1 (a)	$(\frac{QS}{3} \text{ est}((R \times R)^\circ)) \frac{\text{choose}}{3} \text{ est}(R^\circ)$	$\text{est}((R \times R)^\circ) \frac{\text{choose}}{3} \text{ est}(R^\circ) \sqsubseteq E(\text{choose}) \text{ est}(R^\circ)$
2 (b)	$\sqsubseteq (\frac{QS}{3} \text{ est}((R \times R)^\circ)) \frac{\text{choose}}{3} \text{ est}(R^\circ)$	$\frac{QS}{3} \frac{\text{choose}}{3} = \frac{QS}{3} E(\text{choose})$
3 (c)	$\sqsubseteq (\frac{QS}{3} \frac{\text{choose}}{3} \text{ est}(R^\circ))$	

§7.4. Shortest paths on a cylinder

(7.4a)

	on $L = \text{list}^+$, 10.3.3	on Vec	note
1 F	$F(A, X) = A + A \times X$	$F(A, A[m]) = A[1] + A[m+1]$	A square alone, or in front of the path so far.
2 the input	$L(N\ A)$	$A[m][n] \triangleq (A[n])[m]$	m columns of n squares, outermost index first — the matrix itself.
3 a path	$L\ A$, a non-empty list	$A[1] \triangleq A$, $A[m+1] \triangleq A \times A[m]$	Through m columns, m squares: the count is in the type. A one-square path is the square; $\text{Vec}(m)$ is the functor $X \mapsto X[m]$.
4 the paths into a row	$P(L\ A)$	$A[p][m]$, $p = 3^{m-1}$	Three moves per column crossed, so this count is in the type too; a set forgets it.
5 α	$[\text{wrap}, \text{cons}]$ $F(A, LA) \rightarrow LA$	$\text{wrap} = 1 : A \rightarrow A[1]$ $\text{cons} : A \times A[m] \rightarrow A[m+1]$	wrap is an identity; cons is only an iso, so it stays a bead.
6 moves	$NA \rightarrow PNA$	$X[n] \rightarrow X[3][n]$	$\text{moves}(x) = (\text{up}(x), x, \text{down}(x))$, a $3 \times n$ matrix.
7 trans	$PNX \rightarrow NPX$	$X[3][n] \rightarrow X[n][3]$	The transpose.
8 union	$PPX \rightarrow PX$	$\text{concat} : X[j][k] \rightarrow X[jk]$	The j rows laid end to end; j is 3 inside gen and n inside paths .
9 zip	$F(NA, NB) \rightarrow NF(A, B)$	$F(A[n], B[n]) \rightarrow F(A, B)[n]$	Unchanged.
10 cp	$F(A, PB) \rightarrow P(F(A, B))$	$A \times B[p] \rightarrow (A \times B)[p]$	The square paired with each of the p candidates.
11 $\text{est}(R) \triangleq \exists n (\in \text{PPX}) \rightarrow X$		$X[k] \rightarrow X$, $\exists : X[k] \rightarrow X$	A component that is R -below every component; $\exists$ is “is some component” and $\in$ its converse.
12 $R \triangleq \text{sum} \leq \text{sum}^\circ$	$L\ \text{Nat} \rightarrow L\ \text{Nat}$	$\text{Nat}[m] \rightarrow \text{Nat}[m]$	The cost of a path, which the cheapest minimises — one order per length m .
13 gen	$F(1, \text{moves trans } N(\text{union}))$ $\text{zip } N(\text{cp } P(\alpha))$ $F(NA, N(P(LA)))$ $\rightarrow N(P(LA))$	$F(1, \text{moves trans } \text{Vec}(n)(\text{concat}))$ $\text{zip } \text{Vec}(n)(\text{cp } \text{Vec}(3p)(\text{cons}))$ $F(A[n], A[n][p][m]) \rightarrow A[n][3p][m+1]$	The same composite without α ; the type shown is the cons side, the wrap side is $1 : A[n] \rightarrow A[n]$.
14 (gen)	$L(N\ A) \rightarrow N(P(L\ A))$	$A[m][n] \rightarrow A[n][p][m]$	Columns in, rows out.
15 $\text{paths} \triangleq (\text{gen})$ concat	(gen) setify union $L\ N\ \text{Nat} \rightarrow P(L\ \text{Nat})$	$A[m][n] \rightarrow A[np][m]$	The n rows of p paths laid end to end: every path of the cylinder, in one row.
16 Q , Section 7.4.3	$F(1, \text{moves trans } N(\text{est}(R)))$ $\text{zip } N(\alpha)$	$F(1, \text{moves trans } \text{Vec}(n)(\text{est}(R)))$ $\text{zip}, F(A[n], A[n][m]) \rightarrow A[n][m+1]$	The cheapest of three; no p anywhere. $(Q) : A[m][n] \rightarrow A[n][m]$ has the type of a transpose.
17 the specification paths $\text{est}(R)$	$L\ N\ \text{Nat} \rightarrow L\ \text{Nat}$	$A[m][n] \rightarrow A[m]$, $\text{est}(R) : X[np] \rightarrow X$	A cheapest path from the entry side to the exit side.

§7.4.1. $\text{gen} \triangleq (1 \times (\text{moves trans } \text{Vec}(n)(\text{concat}))) \text{ zip } \text{Vec}(n)(\text{cp } \text{Vec}(3p)(\text{cons}))$

(7.4.1a)



(7.4.1b)

a row is the outer index, a cell one path

$u = ((1,2,3,4), C)$

$C = 5$

6

7

8

$(1 \times (\text{moves trans Vec}(n)(\text{concat})))$

$\text{moves}(C)$

$= 6 \ 7 \ 8 \ 5$

5 6 7 8

8 5 6 7

$\text{trans}(\text{that})$

$= 6 \ 5 \ 8$

7 6 5

8 7 6

5 8 7

$\text{Vec}(n)(\text{concat})(\text{that}) = \text{that}$

$\text{zip}(\text{that})$

$= (1, 6 \ 5 \ 8)$

(2, 7 6 5)

(3, 8 7 6)

(4, 5 8 7)

$\text{Vec}(n)(\text{cp})(\text{that})$

$= 1 \ 6 \ 2 \ 7 \ 3 \ 8 \ 4 \ 5$

1 5 2 6 3 7 4 8

1 8 2 5 3 6 4 7

: $A[n] \times A[n]$

one candidate per row, one square long:

a column of squares

1 keeps the column, the candidates move down, unmoved, up: a $3 \times n$ matrix

: $A[3][n]$

the transpose: row k gets rows $k-1, k, k+1$

: $A[n][3]$

$\text{concat}: X[3][1] \rightarrow X[3]$ is 1

each row: its square, and its candidates

: $(A \times A[3])[n]$

cp pairs the square with each candidate:

row k of that is the k -th 3×2 block, left to right; each line a two-square path

: $A[n][3][2]$

§7.4.2. gen is an F-algebra; $(\text{gen}): \text{cons } (\text{gen}) = (1 \times (\text{gen})) \text{gen}$

(7.4.2a)



the $[m+1]$ bracket puts the column back on the matrix, then the fold reads all of it

the fold reads the rest under F , then one gen puts the column in front

(7.4.2b)

$xs = 1 \ 2 \ 3 \ 4$

5 6 7 8

: $A[m][n], m=2$

$(\text{gen})(xs) = \text{gen}((1,2,3,4), (\text{gen})(5,6,7,8))$

$(\text{gen})(5,6,7,8)$

$= 5$

6

7

8

one column: each square its own one-square path

: $A[n][1][1]$

both halves are in (7.4.1b): the tail folds to the column 5 6 7 8, one path per row and each of them one square long, and gen on it is that walk.

§7.4.3. $Q \triangleq (1 \times (\text{moves trans Vec}(n)(\text{est}(R)))) \text{ zip}, A[n] \times A[n][m] \rightarrow A[n][m+1]$

(7.4.3a)



(7.4.3b)

$u = ((1,2,3,4), C)$
 $C = 5$
 6
 7
 8
 $(1 \times (\text{moves trans Vec}(n)(\text{est}(R))))$
 $\text{moves}(C)$
 $= 6 \ 7 \ 8 \ 5$
 $5 \ 6 \ 7 \ 8$
 $8 \ 5 \ 6 \ 7$
 $\text{trans}(\text{that})$
 $= 6 \ 5 \ 8$
 $7 \ 6 \ 5$
 $8 \ 7 \ 6$
 $5 \ 8 \ 7$
 $\text{Vec}(n)(\text{est}(R))(\text{that})$
 $= 5 \ 5 \ 6 \ 5$
 $\text{zip}(\text{that})$
 $= 1 \ 5$
 $2 \ 5$
 $3 \ 6$
 $4 \ 5$

$: A[n] \times A[n][m]$
 one path per row, one square long

 1 keeps the column, the paths move down, unmoved, up: a $3 \times n$ matrix

 $: A[3][n][m]$
 the transpose: row k gets rows $k-1, k, k+1$

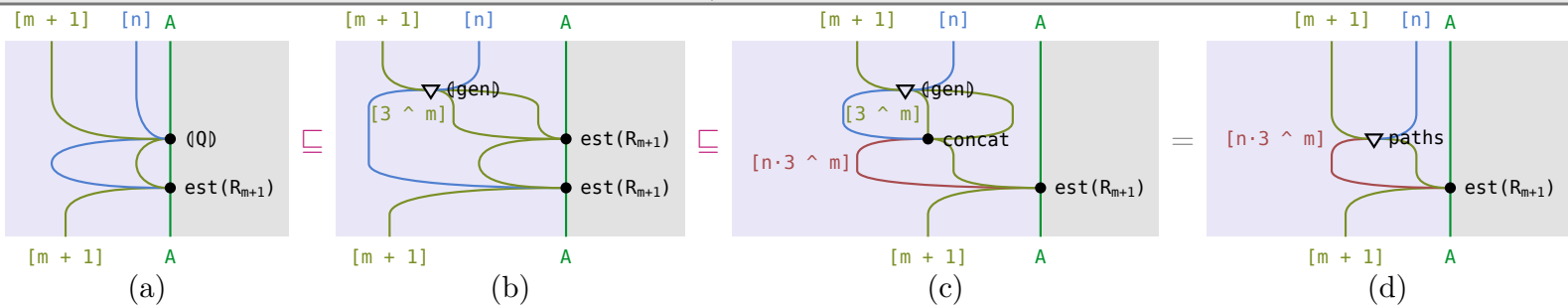
 $: A[n][3][m]$
 the cheapest into each row – chosen BEFORE
 $: A[n][m]$ the new square is put in front each row: its square, and the one predecessor that survived; the $[m+1]$ bracket is cons

 $: A[n][m+1]$

§7.4.4. paths $\text{est}(R) \sqsubseteq (Q) \text{ est}(R)$

$(\forall i. 1 \sqsubseteq R_i) \wedge (\forall i. R_i R_i \sqsubseteq R_i) \wedge (\forall i. (1 \times R_i) \text{ cons} \sqsubseteq \text{cons } R_{i+1}) \Rightarrow (Q) \text{ est}(R_{m+1}) \sqsubseteq \text{paths est}(R_{m+1})$ (7.4.4a)

a cheapest of all **np** paths of the cylinder is beaten by the greedy fold's one path per row and then a cheapest of those n , which costs $O(n \times m)$.



(a)	$(Q) \text{ est}(R_{m+1})$	$(\forall i. 1 \sqsubseteq R_i) \wedge (\forall i. R_i R_i \sqsubseteq R_i) \wedge (\forall i. (1 \times R_i) \text{ cons} \sqsubseteq \text{cons } R_{i+1})$ $\Rightarrow (Q) \sqsubseteq (Qgen) \text{ Vec}(n)(\text{est}(R_{m+1}))$
(b)	$\sqsubseteq (Qgen) \text{ Vec}(n)(\text{est}(R_{m+1})) \text{ est}(R_{m+1})$	$S S \sqsubseteq S$ $\Rightarrow \text{Vec}(j)(\text{est}(S)) \text{ est}(S) \sqsubseteq \text{concat est}(S)$
(c)	$\sqsubseteq (Qgen) \text{ concat est}(R_{m+1})$	$f g = f g$
(d)	$= \text{paths est}(R_{m+1})$	

$S S \sqsubseteq S \Rightarrow \text{Vec}(j)(\text{est}(S)) \text{ est}(S) \sqsubseteq \text{concat est}(S)$, R transitive

a cheapest of each of the j rows and then a cheapest of those j is a cheapest of all jk entries laid end to end.

$(\forall i. 1 \sqsubseteq R_i) \wedge (\forall i. R_i R_i \sqsubseteq R_i) \wedge (\forall i. (1 \times R_i) \text{ cons} \sqsubseteq \text{cons } R_{i+1}) \Rightarrow (Q) \sqsubseteq (Qgen) \text{ Vec}(n)(\text{est}(R_{m+1}))$, R reflexive, transitive and monotonic

the one path per row the greedy fold keeps is one of the p that $(Qgen)$ generates for that row, and a cheapest of them.

$$R_m \ R_m \sqsubseteq R_m \wedge (1 \times R_m) \text{ cons } \sqsubseteq \text{ cons } R_{m+1} \Rightarrow (1 \times \text{Vec}(n)(\text{est}(R_m))) \ Q \sqsubseteq \text{gen } \text{Vec}(n)(\text{est}(R_{m+1})) \quad (7.13)$$

choosing a cheapest of each square's $\mathbf{p}$ paths before the column is extended is no better than extending first and choosing among the $3\mathbf{p}$.



(a)	$\text{gen } \text{Vec}(n)(\text{est}(R_{m+1}))$	(7.13); zip, trans, moves lax natural
(b)	$\sqsubseteq (1 \times \text{Vec}(n)(\text{est}(R_m))) \ Q$	

$$(7.13) \text{ on } \text{Vec}: (1 \times R_m) \text{ cons } \sqsubseteq \text{ cons } R_{m+1} \Rightarrow (1 \times \text{est}(R_m)) \text{ cons } \sqsubseteq \text{cp } \text{Vec}(k)(\text{cons}) \text{ est}(R_{m+1}), R \text{ monotonic}$$

putting the new square in front of every one of the $\mathbf{p}$ candidates and then choosing a cheapest is beaten by choosing a cheapest first and putting the square in front of that one.

§7.5. The security van problem

(7.5a)

	definition	type	example
1	$R \triangleq \text{length} \leq \text{length}^\circ$	$[[A]] \rightarrow [[A]]$	$[[a, b, c]]$ to $[[a], [b, c]]$, and not back: $1 \leq 2$.
2	$\text{ceiling} \triangleq \frac{\text{prefix sum}}{\ni} \text{est}(\geq)$	$[A] \rightarrow \text{Int}$	$\text{ceiling}[a, b]$ is the largest of $0, a, a+b$.
3	$\text{floor} \triangleq \frac{\text{prefix sum}}{\ni} \text{est}(\leq)$	$[A] \rightarrow \text{Int}$	$\text{floor}[a, b]$ is the smallest of $0, a, a+b$, so $[a, b]$ carries $\text{ceiling} - \text{floor}$ cash.
4	secure the coreflexive on x with $\text{bmax}(\text{ceiling } x, \text{ceiling } x - \text{floor } x) \leq N$	$[A] \rightarrow [A]$	secure keeps $[a]$ exactly when $-N \leq a \leq N$.
5	ok the coreflexive on (a, xs) with xs non-empty and $[a] \# \text{head}(xs)$ secure	$A \times [[A]] \rightarrow A \times [[A]]$	ok keeps $(a, [[b], [c]])$ exactly when $[a, b]$ is secure.
6	$\text{new} \triangleq (\text{wrap} \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{new}(a, [[b], [c]]) = [[a], [b], [c]]$.
7	$\text{glue} \triangleq (1 \times \text{cons}^\circ) \text{ assocl } (\text{cons} \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{glue}(a, [[b], [c]]) = [[a, b], [c]]$.
8	$\text{old} \triangleq (1 \times \text{cons}^\circ) \text{ assocl } ((\text{cons secure}) \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{old}(a, [[b], [c]]) = [[a, b], [c]]$ when $[a, b]$ is secure, and nothing otherwise.
9	$\text{partition} = ([\text{nil}, \text{new} \cup \text{glue}])$ cutting the transactions into non-empty segments is one pass along them that either opens a segment for the transaction in hand or puts it on the front of the segment already open	$[A] \rightarrow [[A]]$	$\text{partition}[a, b]$ gives $[[a], [b]]$ and $[[a, b]]$.
10	$S \triangleq [\text{nil}, \text{new} \cup \text{old}]$	$F[[A]] \rightarrow [[A]]$	$S(a, [[b], [c]])$ gives $[[a], [b], [c]]$, and $[[a, b], [c]]$ when $[a, b]$ is secure.
11	$(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow$ $\text{partition list}(\text{secure}) = (S)$ cutting the transactions into non-empty segments every way and keeping the cuts whose every segment is secure is the one fold whose algebra is $[\text{nil}, \text{new} \cup \text{old}]$	$[A] \rightarrow [[A]]$	Both take $[a, b]$ to $[[a], [b]]$ — each $[a]$ is secure — and to $[[a, b]]$ when $[a, b]$ is.
12	$H \triangleq (\text{head prefix}^\circ \text{ head}^\circ) \cup (\text{nil}^\circ \text{ nil})$	$[[A]] \rightarrow [[A]]$	$[[a], [b, c]]$ to $[[a, b], [c]]$: $[a]$ is a prefix of $[a, b]$.
13	$R; H \triangleq R \circ (R^\circ \Rightarrow H)$	$[[A]] \rightarrow [[A]]$	$[[a, b, c]]$ to $[[a], [b, c]]$ by $ R $, and $[[a], [b, c]]$ to $[[a, b], [c]]$ by $R \circ H$.
14	$ R \triangleq R \circ \neg R^\circ$	$[[A]] \rightarrow [[A]]$	$[[a, b, c]]$ to $[[a], [b, c]]$: $1 < 2$. The strict part R splits into: $R; H = R \cup (R \circ H)$.
15	the specification $\frac{\text{partition list}(\text{secure})}{\ni} \text{est}(R)$	$[A] \rightarrow [[A]]$	$[a, b]$ gives $[[a, b]]$ when $[a, b]$ is secure, and $[[a], [b]]$ otherwise.

§7.5.1. secure prefix $\ni$ prefix secure

(7.5a)

(7.5.1a)



$$\text{secure } \langle \text{ceiling}, \text{ceiling-floor} \rangle \text{ bmax} = \langle \text{ceiling}, \text{ceiling-floor} \rangle \text{ bmax } (\leq N)$$

a stretch passes **secure** before the test exactly where the test's own value passes $\leq N$ after it, which is (7.5a.4)'s $\text{bmax}(\text{ceiling } x, \text{ceiling } x - \text{floor } x) \leq N$

Exercise 7.49



$$\text{secure prefix} \sqsubseteq \text{prefix secure}$$

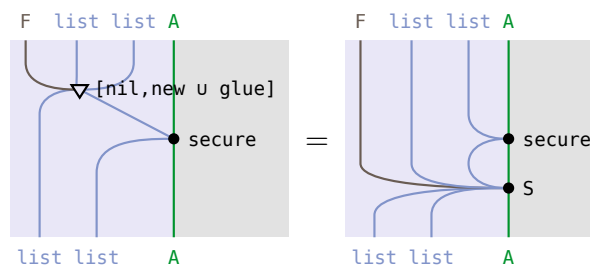
every pair **secure** then **prefix** gives, **prefix** then **secure** gives too — a prefix of a secure stretch is itself secure, which is the prefix-closure
B&dM p.185 names

§7.5.2. partition $\text{list}(\text{secure}) = ([\text{nil}, \text{new } u \text{ old}])$

(7.5.2a)

definition	Hinze–Marsden
1 $\text{new}^{\Delta}(\text{wrap} \times 1) \text{ cons}$ wrap makes the transaction a segment on its own, and cons puts that segment at the front of the schedule: the van is called	
2 $\text{glue}^{\Delta}(1 \times \text{cons}^{\circ}) \text{ assocl } (\text{cons} \times 1) \text{ cons}$ cons° splits the schedule into its first segment and the rest, the first cons puts the transaction at the front of that segment, and the second puts the segment back	
3 $\text{old}^{\Delta}(1 \times \text{cons}^{\circ}) \text{ assocl } ((\text{cons } \text{secure}) \times 1) \text{ cons}$ glue with secure on the segment the transaction has just joined: the van stays away only where the longer segment still passes	

(7.5.2b)



$$(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow [\text{nil}, \text{new } u \text{ glue}] \text{ list}(\text{secure}) = F(\text{list}(\text{secure})) S$$

testing every segment for security after the algebra has run is testing it before it ran: a segment **glue** has extended is secure exactly where **old** would have let it be, and the one transaction **new** sets out on its own is secure whatever it is

(7.5.1a)

$(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow \text{partition list}(\text{secure}) = \langle S \rangle$ Exercise 7.51	
cutting the transactions into non-empty segments every way and keeping the cuts whose every segment is secure is the one fold whose algebra is $[\text{nil}, \text{new} \cup \text{old}]$	
step	Hinze–Marsden
partition list(secure) the specification: cut the transactions every way, then keep the cuts whose every segment is secure	
$= \langle [\text{nil}, \text{new} \cup \text{glue}] \rangle \text{list}(\text{secure})$ partition is the fold that either opens a segment for the transaction in hand or puts it on the front of the segment already open	
$= \langle [\text{nil}, \text{new} \cup \text{old}] \rangle$ fusion: the condition above moves $\text{list}(\text{secure})$ inside that fold, which is what turns glue into old	

(7.5.2c)



$$(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow \text{partition list}(\text{secure}) = \langle S \rangle$$

cutting the transactions every way and then keeping the cuts whose every segment is secure is one pass along them that either calls the van or extends the open segment while it stays secure

§7.5.3. $(1 \times R) \text{new} \sqsubseteq (\text{new} \cup \text{old}) R$

$(1 \times R) \text{new} \sqsubseteq (\text{new} \cup \text{old}) R$ (7.14)	
calling the van for the transaction on a no-longer schedule gets no further than calling it on this one and shortening the schedule afterwards	
step	Hinze–Marsden
$(1 \times R) \text{new}$ the left side of (7.14) — (7.5a.1), (7.5a.6)	
$= (\text{wrap} \times R) \text{cons}$ $\text{new} \triangleq (\text{wrap} \times 1) \text{cons} \text{ — } (\text{wrap} \times 1) \text{cons} \text{ — and } (\text{wrap} \times 1) (1 \times R) = (\text{wrap} \times R)$	
$\sqsubseteq \text{new} R$ cons is monotonic on R — $(1 \times R) \text{cons} \sqsubseteq \text{cons} R$, consing onto a no-longer schedule leaves it no longer — and $\text{new} \sqsubseteq \text{new} \cup \text{old}$	

§7.5.4. $H \triangleq (\text{head prefix}^\circ \text{head}^\circ) \cup (\text{nil}^\circ \text{nil})$

(7.5.4a)



§7.5.5. $(1 \times (R;H)) \text{new} \sqsubseteq (\text{new } u \text{ old})(R;H)$

(7.18)



whatever schedule the van is called on, the result's first segment is the one transaction $[a]$, so any two results of **new** on that transaction stand in H

$$(1 \times (R;H)) \text{new} \sqsubseteq (\text{new } u \text{ old})(R;H) \quad (7.16)$$

calling the van for the transaction on a $R;H$ -better schedule gets no further than calling it on this one and bettering the whole schedule afterwards

step	Hinze–Marsden
$(1 \times (R;H)) \text{new}$ the left side of (7.16)	
$\sqsubseteq (1 \times R) \text{new}$ $R;H \sqsubseteq R$ — (7.5a.13)	
$\sqsubseteq \text{new } R \cap \text{new } H$ (7.14) as far as its new R line — (7.14) $(1 \times R) \text{new} \sqsubseteq (\text{new } u \text{ old}) R$ — and (7.18)	
$= \text{new } (R \cap H)$ new is a map, and a map distributes over $\cap$	
$\sqsubseteq \text{new } (R;H)$ $X \cap Y \sqsubseteq X;Y$, and $\text{new} \sqsubseteq \text{new } u \text{ old}$	

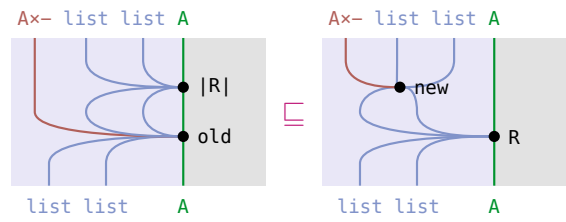
§7.5.6. $(1 \times (R;H)) \text{old} \sqsubseteq (\text{new } u \text{ old})(R;H)$

(7.19)



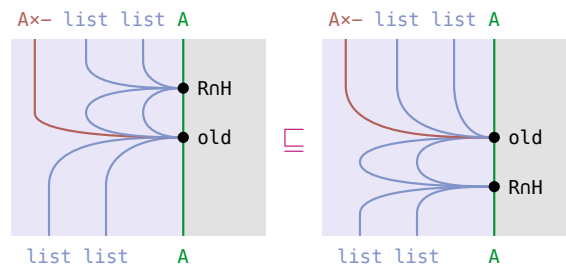
old leaves the transaction **[a]** at the front of the first segment, and **[a]** is what **new** makes that segment, so the two first segments are **prefix-related** whatever the schedules were

(7.20)



old keeps the schedule's length, so a strictly shorter one still comes out no longer than the one the van's own segment lengthens

(7.21)



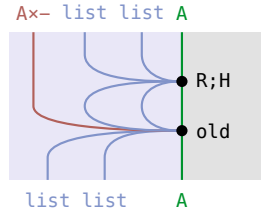
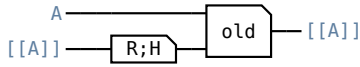
on a tie the one first segment is a prefix of the other, so prefix-closure of **secure** — Exercise 7.49 — lets **old** fire on this side too, and it keeps both the length and the prefix

$$(1 \times (R;H)) \text{ old} \sqsubseteq (\text{new} \cup \text{old}) (R;H) \quad (7.17)$$

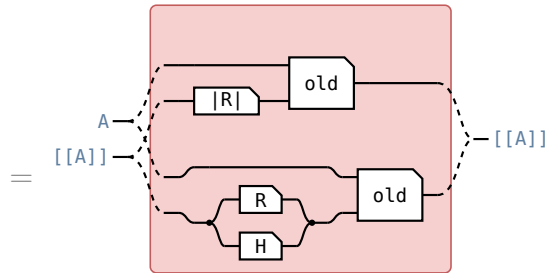
gluing the transaction onto a better schedule for the rest gets no further than gluing it on, or calling the van, and
bettering the whole schedule after,

circuit — the old branch of each union

Hinze–Marsden

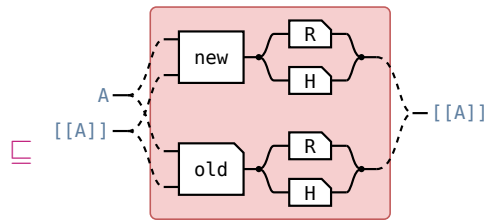
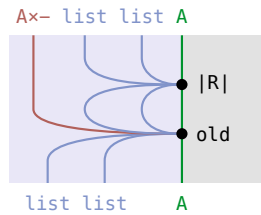


the **old** operand of **new** $\cup$ **old**, in every row



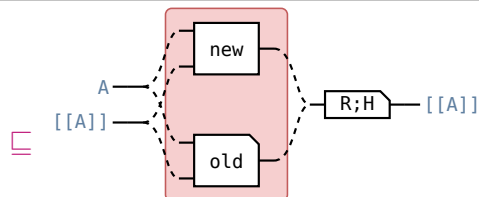
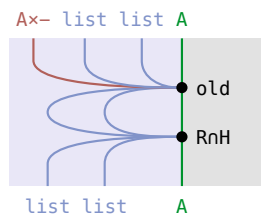
$$(1 \times |R|) \text{ old} \cup (1 \times (RnH)) \text{ old}$$

$R;H = |R| \cup (RnH)$ — (7.5a.14), $\cup$ distributes,



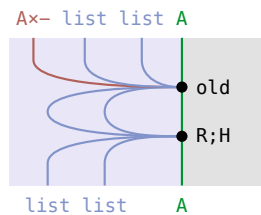
$$\text{new } (RnH) \cup \text{old } (RnH)$$

(7.19) and (7.20) on $|R|$ — (7.19), (7.20) — and (7.21) on RnH — (7.21)



$$(\text{new} \cup \text{old})(R;H)$$

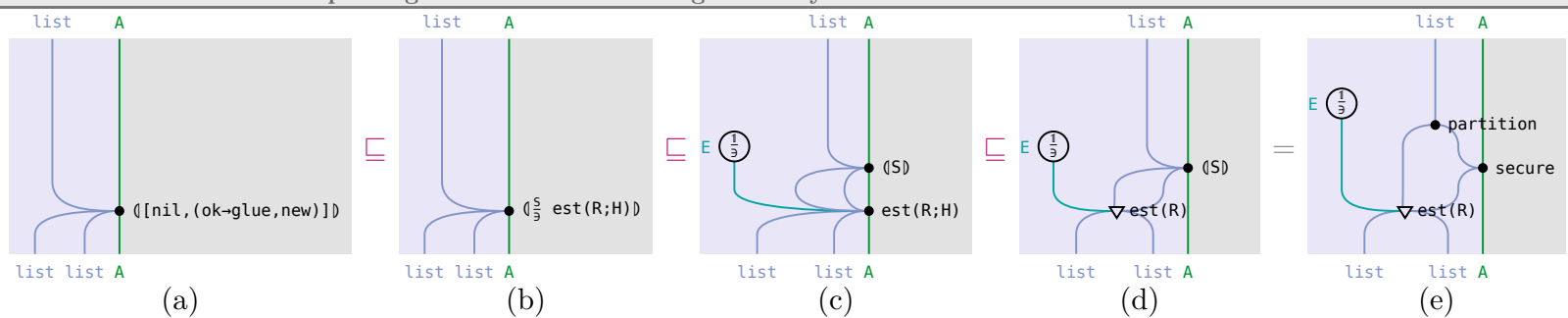
$XnY \sqsubseteq X;Y$, converses



§7.5.7. The derivation

$$(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow ([\text{nil}, (\text{ok} \rightarrow \text{glue}, \text{new})]) \sqsubseteq \frac{\text{partition list}(\text{secure})}{\exists} \text{est}(R) \quad (7.5.7a)$$

the fewest secure segments the transactions can be cut into are one pass along them, the next transaction glued
onto the open segment wherever that segment stays secure and the van called where it does not



(a)	$[\text{nil}, (\text{ok} \rightarrow \text{glue}, \text{new})]$	$[\text{nil}, (\text{ok} \rightarrow \text{glue}, \text{new})] \sqsubseteq \frac{\exists}{\exists} \text{est}(R;H)$
(b)	$\sqsubseteq \frac{\exists}{\exists} \text{est}(R;H)$	$\frac{\exists}{\exists} \text{est}(R;H) \sqsubseteq \frac{\exists(S)}{\exists} \text{est}(R;H)$
(c)	$\sqsubseteq \frac{\exists(S)}{\exists} \text{est}(R;H)$	$R;H \sqsubseteq R$
(d)	$\sqsubseteq \frac{\exists(S)}{\exists} \text{est}(R)$	Exercise 7.51 $(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow \text{partition list}(\text{secure}) = \{S\}$
(e)	$= \frac{\text{partition list}(\text{secure})}{\exists} \text{est}(R)$	